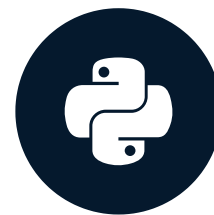


Text classification

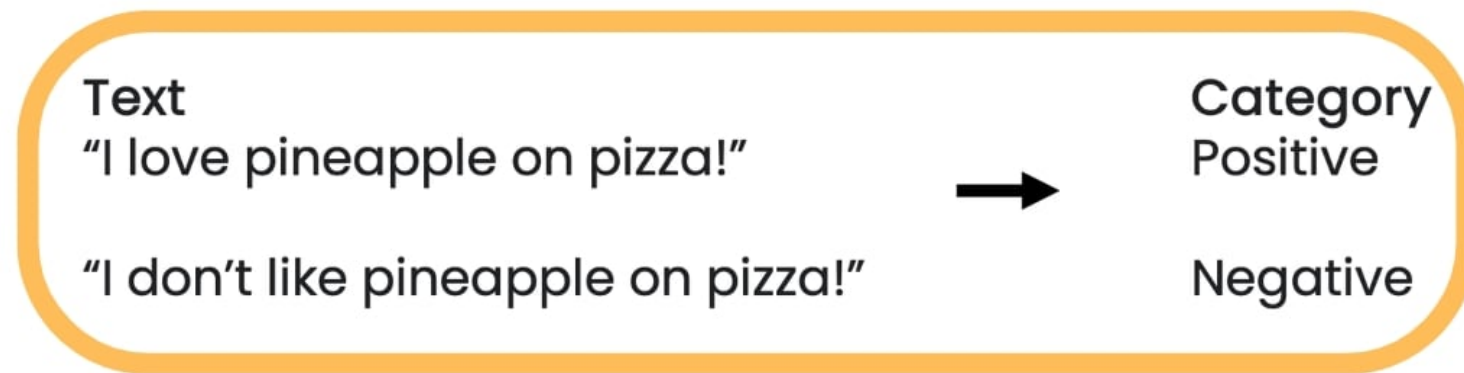
WORKING WITH HUGGING FACE



Jacob H. Marquez
Lead Data Engineer

Text classification: Sentiment analysis

- Labels text based on its emotional tone



- Applications:** Analyzing reviews, tracking social media sentiment



Sentiment analysis: coding example

```
from transformers import pipeline

my_pipeline = pipeline(
    "text-classification",
    model="distilbert-base-uncased-finetuned-sst-2-english"
)

print(my_pipeline("Wi-Fi is slower than a snail today!"))
```

```
[{'label': 'NEGATIVE', 'score': 0.99}]
```

Text classification: Grammatical correctness



- Evaluates text grammar for correctness

Text	Category
"This course is great!"	Acceptable
"Course is gravy."	Unacceptable

- **Applications:** Grammar checkers, language learning tools

Grammatical correctness: coding example

```
from transformers import pipeline

# Create a pipeline for grammar checking
grammar_checker = pipeline(
    task="text-classification",
    model="abdulemonomotoso/English_Grammar_Checker"
)

# Check grammar of the input text
print(grammar_checker("He eat pizza every day."))
```

```
[{'label': 'LABEL_0', 'score': 0.99}]
```

Text classification: QNLI



Question:

"What state is
Hollywood in?"

Category

Premise:

"Hollywood is in
California."



Entailment
(True)

Premise:

"Hollywood is
known for its
movies."



Not Entailment
(False)

- Checks if a premise answers a question
- **Applications:** Q&A systems, fact-checking

QNL: coding example

```
from transformers import pipeline

classifier = pipeline(
    task="text-classification",
    model="cross-encoder/qnli-electra-base"
)

classifier("Where is Seattle located?, Seattle is located in Washington state.")
```

```
[{'label': 'LABEL_0', 'score': 0.997}]
```

Text classification: Dynamic category assignment

- Dynamically assigns categories based on content

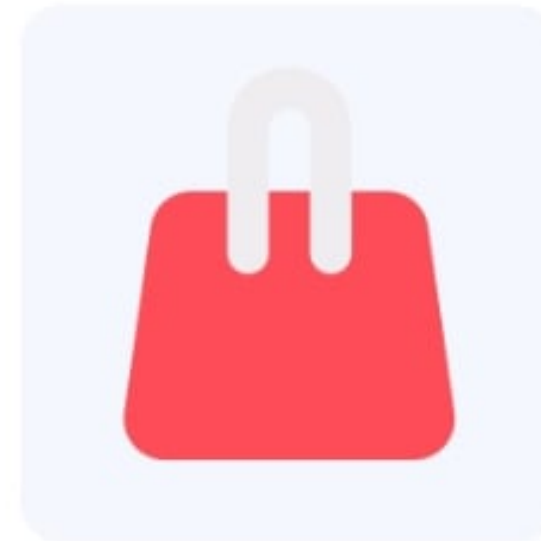
Request:

"I want to know more about your pricing plans."

Categories:

- Sales: → High Confidence (**True**)
- Marketing: → Moderate Confidence (**False**)
- Support: → Low Confidence (**False**)

- **Applications:** Content moderation, recommendation systems



Dynamic category assignment: coding example

```
classifier = pipeline(  
    task="zero-shot-classification",  
    model="facebook/bart-large-mnli")  
  
text = "Hey, DataCamp; we would like to feature your courses in our newsletter!"  
categories = ["marketing", "sales", "support"]  
  
output = classifier(text, categories)  
  
print(f"Top Label: {output['labels'][0]} with score: {output['scores'][0]}")
```

```
Top Label: support with score: 0.8183
```

Challenges of text classification



Ambiguity

Challenges of text classification



Ambiguity



Sarcasm, Irony

Challenges of text classification



Ambiguity



Sarcasm, Irony

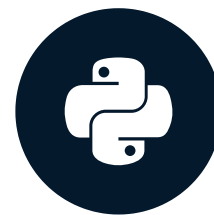


Multilingual

Let's practice!
WORKING WITH HUGGING FACE

Text summarization

WORKING WITH HUGGING FACE



Jacob H. Marquez
Lead Data Engineer

What is summarization?

Original Text

David G. Robinson is a data scientist at the Heap analytics company. He is a co-author of the tidytext R programming language package and the O'Reilly book, Text Mining with R. Robinson has previously worked as a chief data scientist at DataCamp and as a data scientist at Stack Overflow. He was also a data engineer at Flatiron Health in 2019.



Summarized Text

David G. Robinson is a data scientist. He is a co-author of the tidytext R package and the O'Reilly book.

Extractive vs. Abstractive

Extractive:

Selects key sentences from the text

Efficient, needs fewer resources

Lacks flexibility; may be less cohesive

Abstractive:

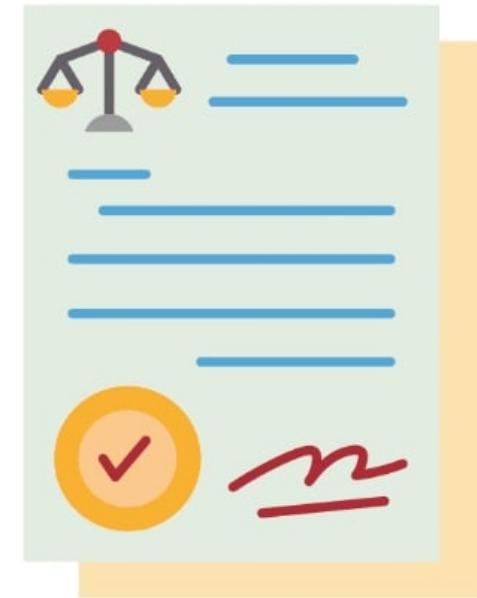
Generates new, rephrased text

Clearer and more readable

Requires more resources and processing

Use cases of extractive summarization

- ☐ **Legal Documents:** Highlights key clauses
- ☐ **Financial Research:** Extracts insights



Use cases of abstractive summarization



- ☐ **News Articles:** Creates concise summaries



- ☐ **Content Recommendations:** Generates compelling descriptions

Extractive summarization in action

```
from transformers import pipeline

# Load the extractive summarization pipeline
summarizer = pipeline("summarization", model="nyamuda/extractive-summarization")
```

```
text = "This is my really large text about Data Science..."
summary_text = summarizer(text)
```

```
print(summary_text[0]['summary_text'])
```

```
"data science is a field that combines mathematics, statistics...."
```

Abstractive summarization in action

```
from transformers import pipeline

# Load the abstractive summarization pipeline
summarizer = pipeline("summarization", model="sshleifer/distilbart-cnn-12-6")

text = "This is my really large text about Data Science..."
summary_text = summarizer(text)
print(summary_text[0]['summary_text'])
```

```
"The global data science platform market is projected  
is projected to reach $140.9 billion by 2025..."
```

Parameters for summarization

- `min_length` & `max_length` : Control summary length

```
summarizer = pipeline(task="summarization", min_length=10, max_length=150)
```

Example Error

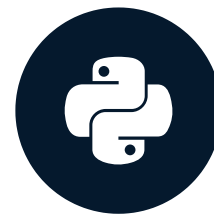
```
Your max_length is set to 150, but your input_length is only 81.  
Since this is a summarization task, where outputs shorter than the input are  
typically wanted, you might consider decreasing max_length manually,  
e.g. summarizer('...', max_length=40)
```

- **Error?** Adjust `max_length` for short inputs

Let's practice!
WORKING WITH HUGGING FACE

Auto Models and Tokenizers

WORKING WITH HUGGING FACE



Jacob H. Marquez
Lead Data Engineer

Pipelines: fast and simple

```
from transformers import pipeline

my_pipeline = pipeline(
    "text-classification",
    model="distilbert-base-uncased-finetuned-sst-2-english"))

print(my_pipeline("Wi-Fi is slower than a snail today!"))
```

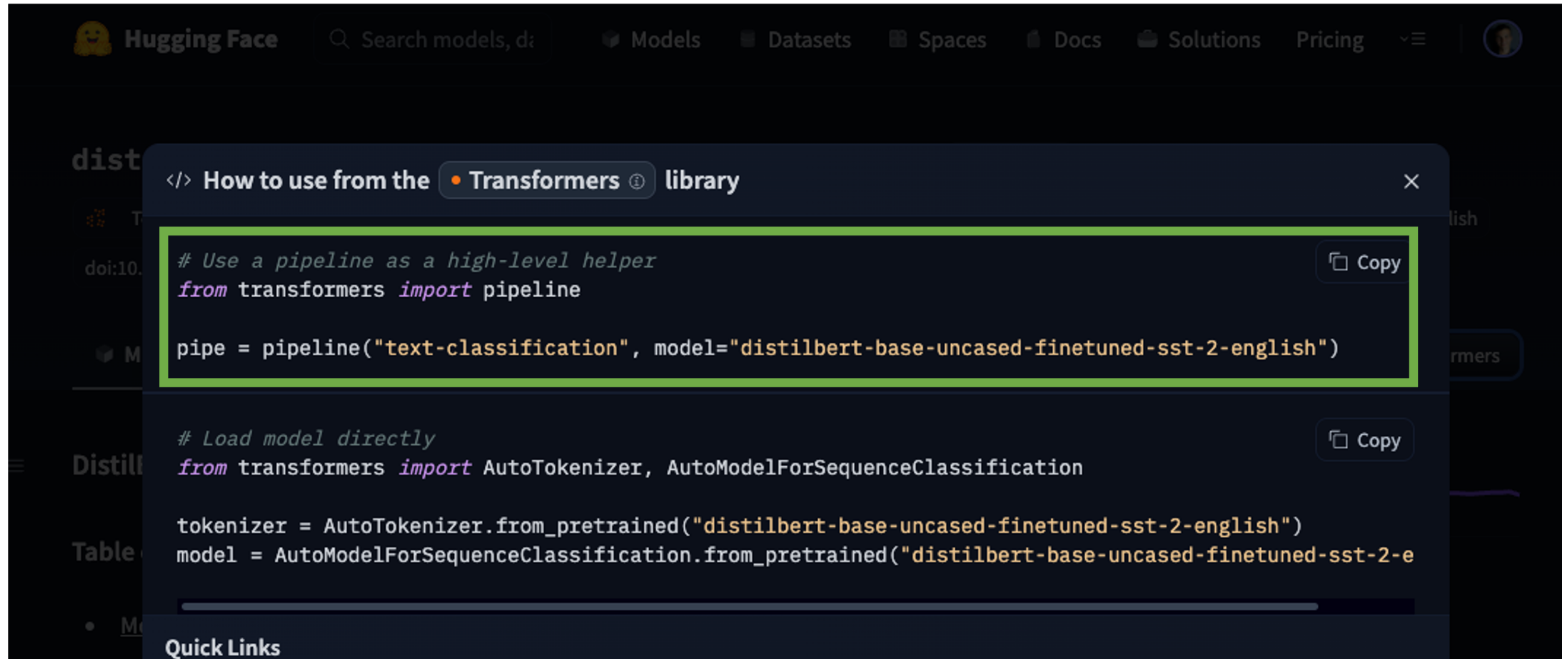
```
[{'label': 'NEGATIVE', 'score': 0.99}]
```

Two ways to use Hugging Face models

The screenshot shows the Hugging Face interface for the model `distilbert-base-uncased-finetuned-sst-2-english`. The top navigation bar includes the Hugging Face logo, a search bar, and links to Models, Datasets, Spaces, Docs, Solutions, Pricing, and a user profile. The model name is displayed with a copy icon and a 'like' button showing 365 likes. Below the name are tags for Text Classification, Transformers, PyTorch, TensorFlow, Rust, ONNX, Safetensors, sst2, glue, and English. Further down are links for the DOI, distilbert, Eval Results, Inference Endpoints, arxiv:1910.01108, and License: apache-2.0. A secondary navigation bar includes Model card (selected), Files, and Community (24). Action buttons for Train, Deploy, and 'Use in Transformers' (highlighted with a green box) are present. The main content area shows the model name 'DistilBERT base uncased finetuned SST-2', a 'Table of Contents' with links to Model Details and How to Get Started With the Model, and a sidebar with 'Downloads last month 21,019,589' and a line graph. Technical details like Safetensors, Model size (67M params), and Tensor type (F32) are also visible.

¹ <https://huggingface.co/distilbert-base-uncased-finetuned-sst-2-english>

Two ways to use Hugging Face models



The screenshot shows the Hugging Face website with a modal window titled "How to use from the Transformers library". The modal displays two code snippets for using the `distilbert-base-uncased-finetuned-sst-2-english` model.

```
# Use a pipeline as a high-level helper
from transformers import pipeline

pipe = pipeline("text-classification", model="distilbert-base-uncased-finetuned-sst-2-english")
```

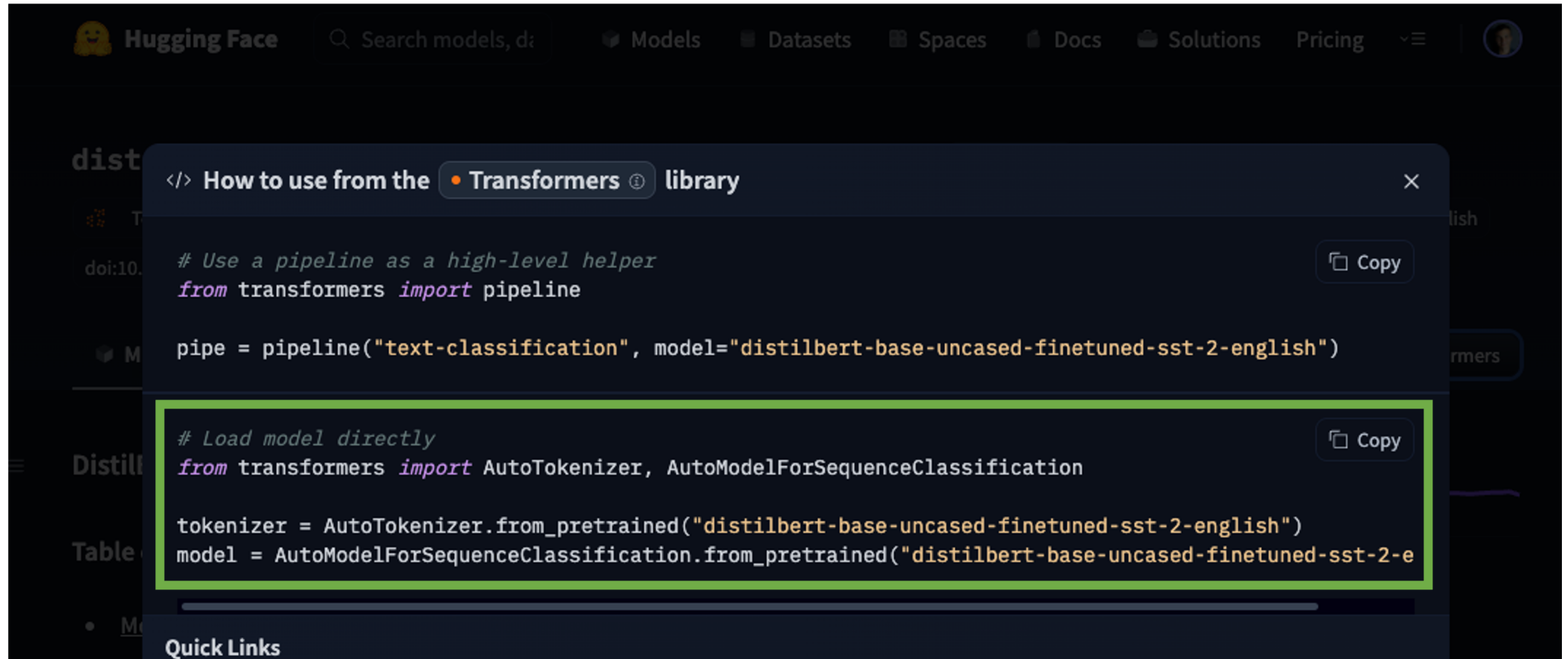
```
# Load model directly
from transformers import AutoTokenizer, AutoModelForSequenceClassification

tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased-finetuned-sst-2-english")
model = AutoModelForSequenceClassification.from_pretrained("distilbert-base-uncased-finetuned-sst-2-e
```

Each code block has a "Copy" button next to it. The background of the website shows the Hugging Face logo, navigation links (Search models, datasets, Models, Datasets, Spaces, Docs, Solutions, Pricing), and a list of models including "distilbert-base-uncased-finetuned-sst-2-english".

¹ <https://huggingface.co/distilbert-base-uncased-finetuned-sst-2-english>

Two ways to use Hugging Face models



The screenshot shows the Hugging Face website with a modal window titled "How to use from the Transformers library". The modal displays two code snippets for using the DistilBERT model. The first snippet uses the `pipeline` function, and the second snippet uses `AutoTokenizer` and `AutoModelForSequenceClassification` to load the model directly. The second snippet is highlighted with a green border.

```
</> How to use from the Transformers library
```

```
# Use a pipeline as a high-level helper
from transformers import pipeline

pipe = pipeline("text-classification", model="distilbert-base-uncased-finetuned-sst-2-english")
```

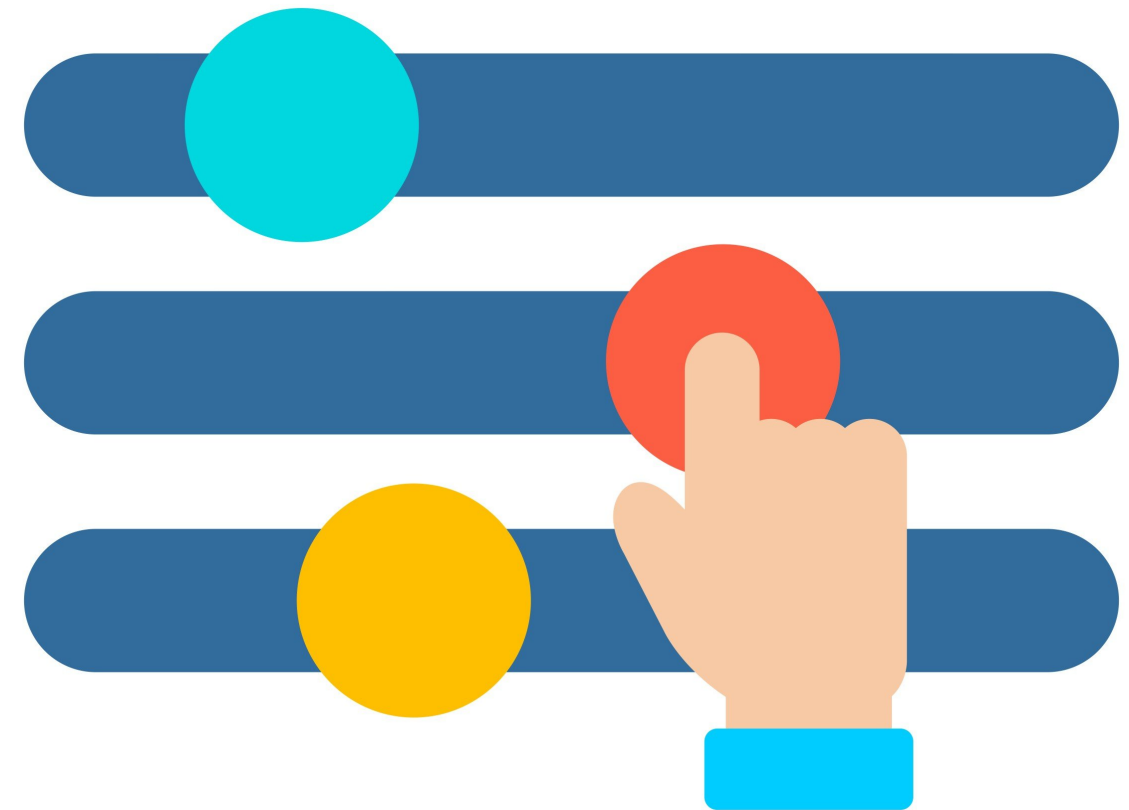
```
# Load model directly
from transformers import AutoTokenizer, AutoModelForSequenceClassification

tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased-finetuned-sst-2-english")
model = AutoModelForSequenceClassification.from_pretrained("distilbert-base-uncased-finetuned-sst-2-e
```

¹ <https://huggingface.co/distilbert-base-uncased-finetuned-sst-2-english>

Auto Classes: flexible and powerful

- **Auto classes:** Flexible access to models and tokenizers
- **More control** over model behavior and outputs
- **Perfect for advanced tasks**
- Pipelines = quick; Auto classes = flexible



AutoModels

- Choose an AutoModel class to directly download a model

```
from transformers import AutoModelForSequenceClassification

# Download a pre-trained text classification model
model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased-finetuned-sst-2-english"
)
```

AutoTokenizers

- Prepare text input data
- Recommended to use the **tokenizer paired with the model**

```
from transformers import AutoTokenizer

# Retrieve the tokenizer paired with the model
tokenizer = AutoTokenizer.from_pretrained(
    "distilbert-base-uncased-finetuned-sst-2-english"
)
```

Tokenizing text with AutoTokenizer

- Tokenizers clean input and split text into tokens

```
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")

# Tokenize input text
tokens = tokenizer.tokenize("AI: Helping robots think and humans overthink:")
print(tokens)
```

```
['ai', ':', 'helping', 'robots', 'think', 'and',  
 'humans', 'over', '##thi', '##nk', ':', '']
```

Different models, different tokenizers

- Our model (distilbert-base-uncased):

```
['ai', ':', 'helping', 'robots', 'think', 'and', 'humans', 'over', '##thi',  
'##nk', ':', ')']
```

- BERT-Base-Cased Tokenizer:

```
['AI', ':', 'Help', '##ing', 'robots', 'think', 'and', 'humans', 'over',  
'##thin', '##k', ':', ')']
```

Building a Pipeline with Auto Classes

```
from transformers import AutoModelForSequenceClassification,  
AutoTokenizer, pipeline  
  
# Download the model and tokenizer  
my_model = AutoModelForSequenceClassification.from_pretrained(  
    "distilbert-base-uncased-finetuned-sst-2-english")  
my_tokenizer = AutoTokenizer.from_pretrained(  
    "distilbert-base-uncased-finetuned-sst-2-english")  
  
# Create the custom pipeline  
my_pipeline = pipeline(  
    task="sentiment-analysis", model=my_model, tokenizer=my_tokenizer)
```

Use Cases for AutoModels and AutoTokenizers

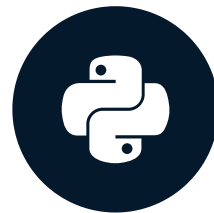
- ☐ Use for more control and customization
- ☐ **Text Preprocessing:** Clean and tokenize for specific use cases
- ☐ **Thresholding:** Prioritize key categories in classification tasks
- ☐ **Complex Workflows:** Control multi-stage processing and integration



Let's practice!
WORKING WITH HUGGING FACE

Document Q&A

WORKING WITH HUGGING FACE



Jacob H. Marquez
Lead Data Engineer

What is document question and answering?

- Answers questions from document content
- Requires a document and a question
- Provides direct or paraphrased answers

Question: "What is the total revenue of Q3?"

Memo to: Finance Department

Date: October 20, 2023

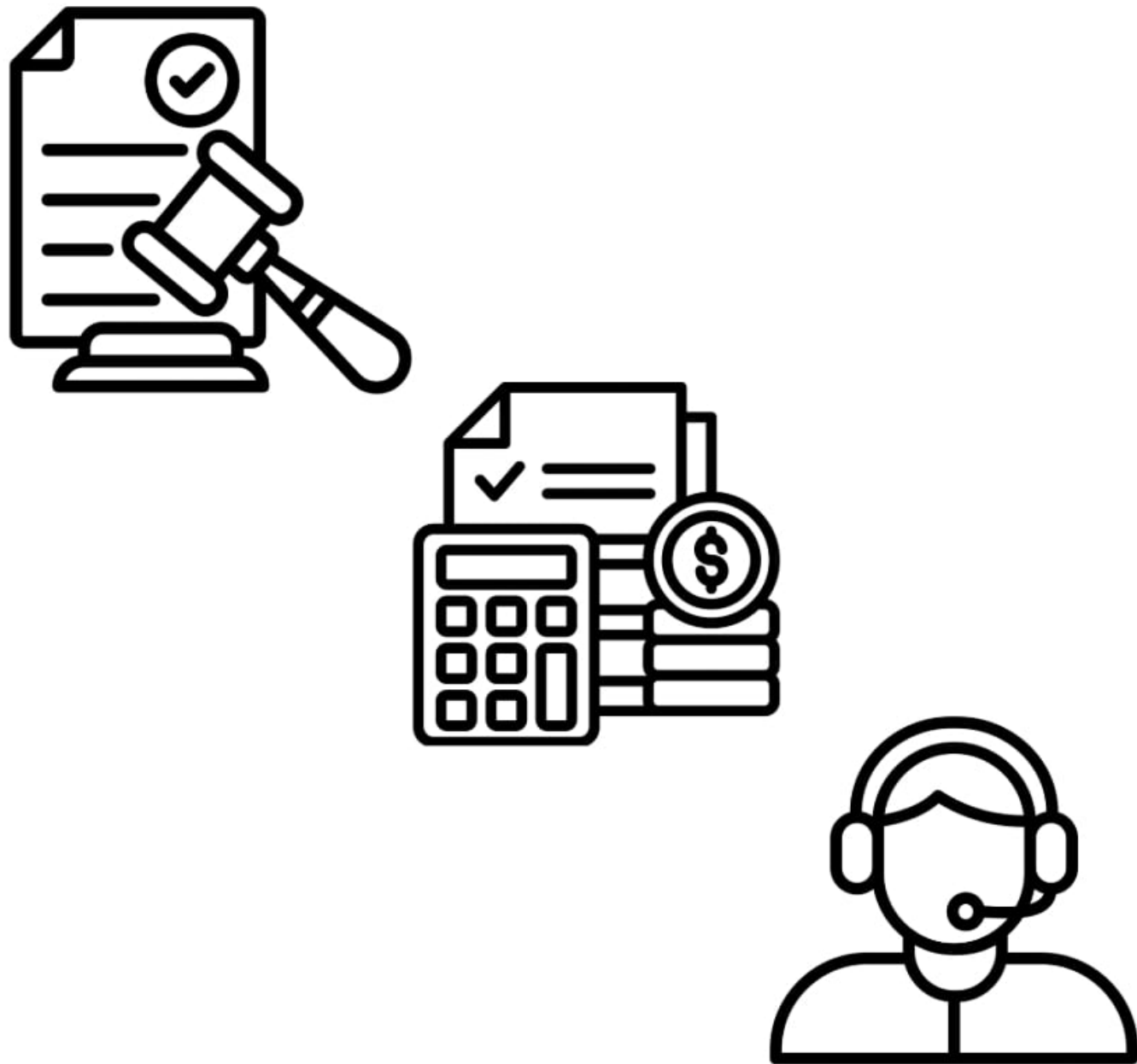
Page 1

Quarterly Financial Update

- Total revenue for Q3 was **\$10.5 million**, marking a **15% increase** from the previous quarter.
- Operating expenses for Q3 totaled **\$4.2 million**, resulting in a net profit of **\$6.3 million**.
- Key drivers of growth included increased product demand and an expanded customer base.

For any questions regarding this update, please contact **the finance team** directly.

Use cases for document Q&A



- ☐ **Legal:** Identify contract clauses
- ☐ **Finance:** Extract key figures
- ☐ **Support:** Retrieve answers from manuals

Automating HR queries with document Q&A

- □ Info stored in `US-Employee_Policy.pdf`
- □ Build a system to extract answers
- □ Save HR time and effort



Extracting text with pypdf

```
from pypdf import PdfReader

# Load the PDF file
reader = PdfReader("US-Employee_Policy.pdf")

# Extract text from all pages
document_text = ""
for page in reader.pages:
    document_text += page.extract_text()
```

```
Welcome to the US Employee Policy document...
```

Creating a Q&A pipeline

```
# Load the question-answering pipeline
qa_pipeline = pipeline(
    task="question-answering",
    model="distilbert-base-cased-distilled-squad")

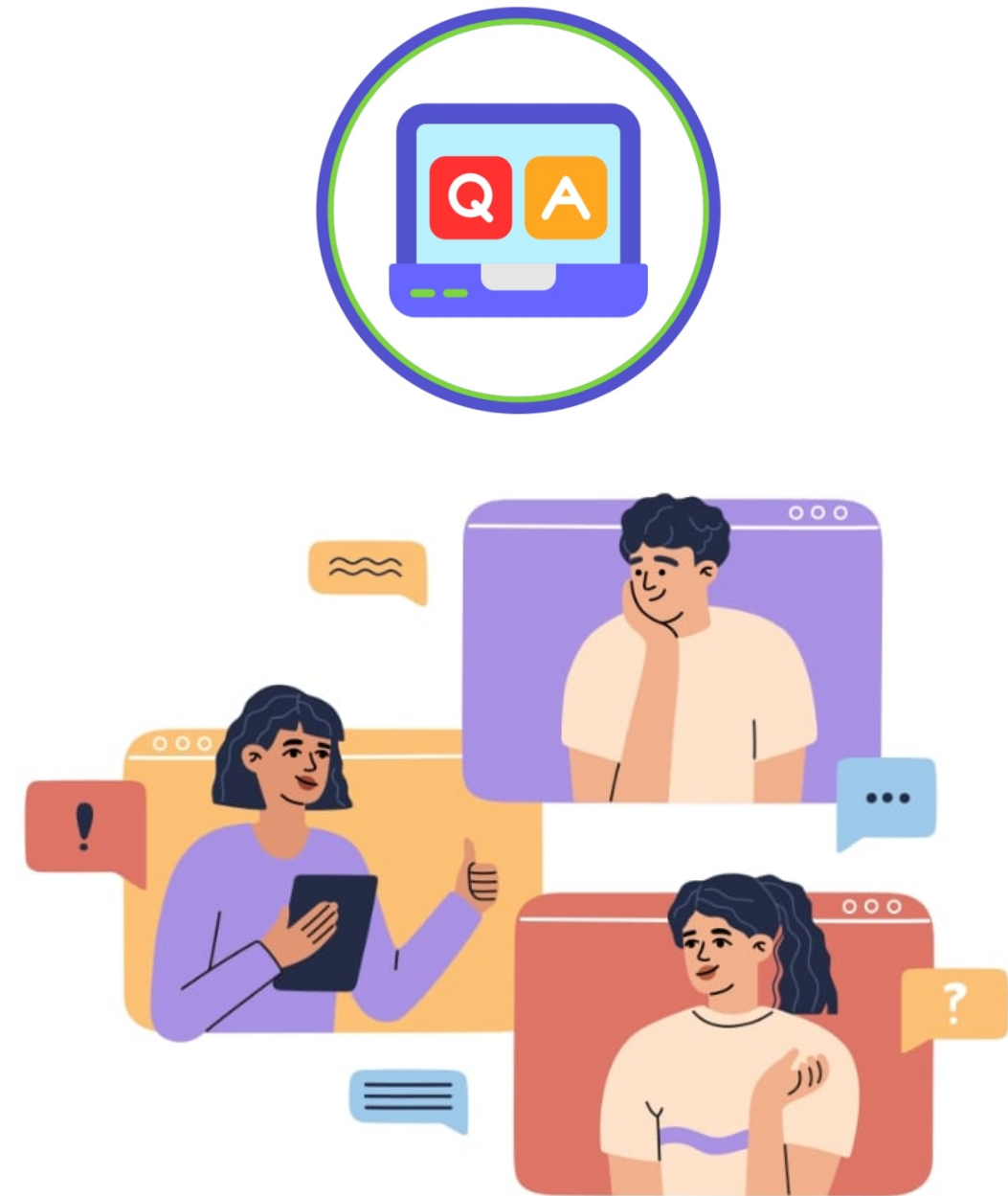
question = "How many volunteer days are offered annually?"

# Get the answer from the QA pipeline
result = qa_pipeline(question=question, context=document_text)
print(f"Answer: {result['answer']}")
```

Answer: 1

Bringing it all together

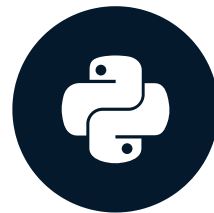
- Use `PdfReader` from `pypdf` to load and read PDF files
- Extract text with `.pages` and `.extract_text()` into `document_text`
- Set up a `question-answering` pipeline
- Pass a `question` and `context` to the pipeline
- Wrap into functions to automate queries



Let's practice!
WORKING WITH HUGGING FACE

Congratulations!

WORKING WITH HUGGING FACE



Jacob H. Marquez
Lead Data Engineer

Chapter 1

- ☐ **Hugging Face Hub:** A platform for discovering AI models and datasets
- ☐ **Pre-trained Models:** create pipelines and save them with `.save_pretrained()`
- ☐ **Datasets:** preprocess data using `.filter()` and `.select()`



Chapter 2



Text classification

- ☐ **Sentiment Analysis:** Understand customer emotions
- ☐ **Grammar Checks:** Identify errors in text
- ☐ **Category Assignment:** Classify requests with scores

```
pipeline(task="text-classification")
```

```
[{'label': 'POSITIVE', 'score': 0.97}]
```



Text summarization

Original Text

David G. Robinson is a data scientist at the Heap analytics company. He is a co-author of the tidytext R programming language package and the O'Reilly book, Text Mining with R. Robinson has previously worked as a chief data scientist at DataCamp and as a data scientist at Stack Overflow. He was also a data engineer at Flatiron Health in 2019.

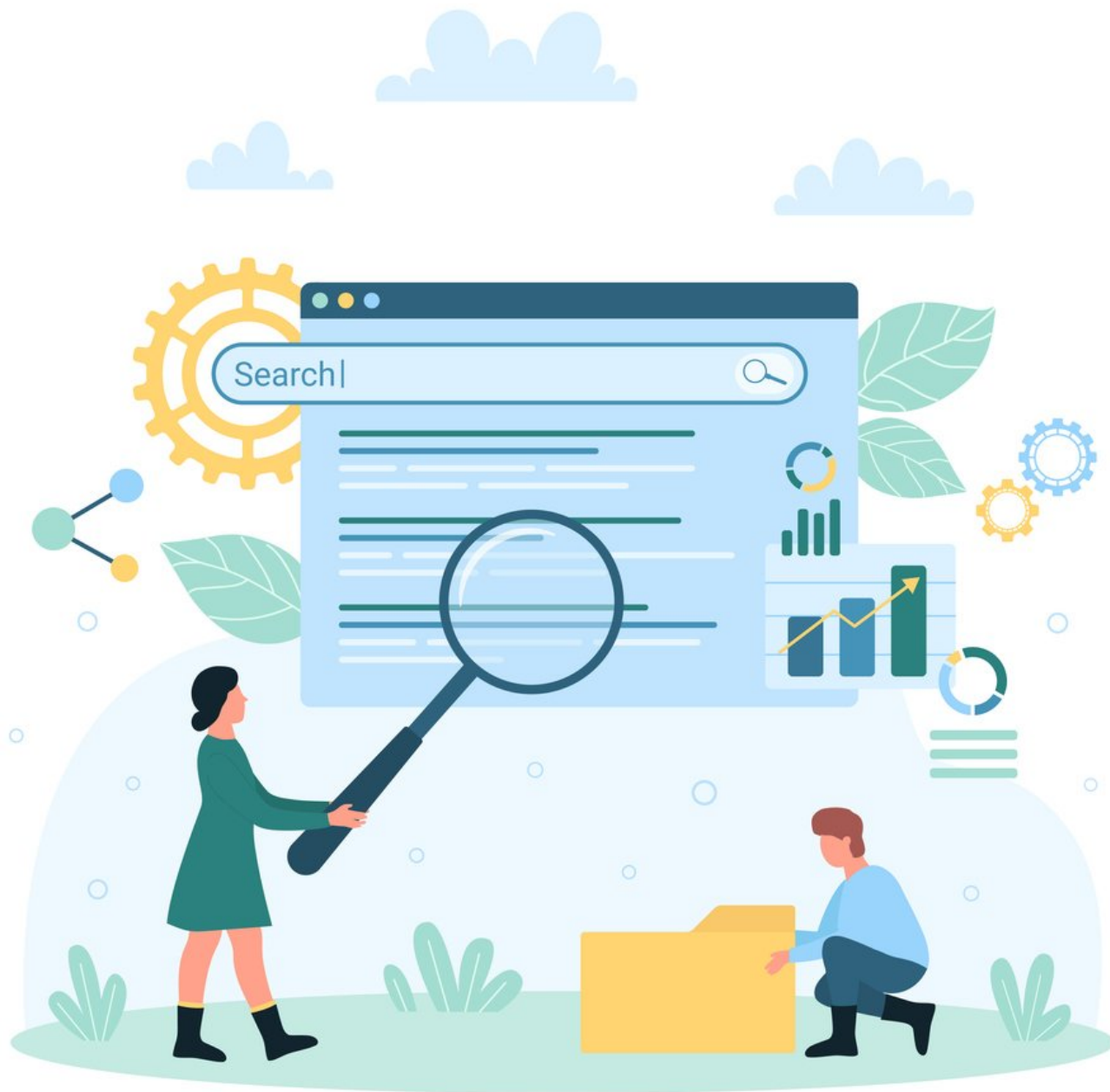


Summarized Text

David G. Robinson is a data scientist. He is a co-author of the tidytext R package and the O'Reilly book.

- **□ Pipeline:** Specify `pipeline(task="summarization")`
- **□ Output Length:** Adjust with `min_length` and `max_length`

Document Q&A



- **PDF Processing:** Use `.pages` and `.extract_text()` from `pypdf` to extract text
- **Q&A Pipeline:** Specify `pipeline(task="question-answering")`

Auto Models and Tokenizers



Congratulations and Thank You!

WORKING WITH HUGGING FACE